

Beacon pool console

Chapter 2 · Practical Lab 2 · Connecting and composing classes

In this lab, you will connect several small C++ classes and build a tiny reusable beacon pool. You will practise deciding when a header needs a forward declaration, when it needs a full class definition, how one class can contain objects of another class, and how class-wide state differs from the state stored inside each object.

This is not a copy-the-code exercise. The public interfaces and the console calls are provided where they help you focus on the object-oriented design. Most member-function definitions and control logic are for you to write progressively.

Duration: 120 minutes. Start from an empty Visual Studio project and build one working stage at a time.

How to use this sheet. For each stage, first read the required behaviour. Then write the definition yourself. A hint is included when the syntax is new, but the completed function body is usually not supplied. Build and run at every checkpoint.

Before you begin

Use a Windows PC with **Visual Studio 2026**, the **Desktop development with C++** workload, Platform Toolset **v145**, C++17, **/W4**, **Debug** and **x64**.

Prerequisites: Practical Lab 1 class basics, constructors, private data, const member functions, references, fixed arrays, loops and simple `if` statements.

What you will practise

- using a forward declaration when a class name is enough;
- including a complete type when another object is stored by value;
- implementing constructors and member-function definitions from an interface;
- using composition to build a larger class from smaller class objects;
- using friendship for a narrow, deliberate access relationship;
- implementing a fixed-capacity request/release pool without dynamic allocation;
- distinguishing per-object pool state from one shared static tally; and
- tracing reuse so released objects are reset before they are used again.

Not required in this lab

You are not required to use inheritance, `new/delete`, smart pointers, templates, STL containers, a graphics library, a game loop, threads or a production-grade handle system. The pool uses a fixed built-in array so that the class relationships remain visible.

The problem

A small game level uses beacons. A receiver can measure the one-dimensional distance to a beacon. During play, a BeaconPool keeps three reusable beacon objects ready. A request activates the first free beacon with a signal strength. Releasing a beacon resets it so that the same slot can be used again.

The lab deliberately separates three questions from Lecture 2: what the compiler needs to know, what private access is allowed, and which object owns the lifetime of another object.

Files you will create

File	Responsibility
Main.cpp	Creates objects, calls the public interfaces and prints the published checks.
Beacon.h / Beacon.cpp	Defines one beacon and its private active/reset behaviour.
Receiver.h / Receiver.cpp	Uses a Beacon by const reference without owning it.
BeaconPool.h / BeaconPool.cpp	Owns three Beacon objects by value and controls their reuse.

What the finished program will show

```
Beacon pool lab

Connection check
Range to beacon: 25

Pool A
Start active: 0
Request 30 -> slot 0
Request 60 -> slot 1
Request 90 -> slot 2
Request 50 -> slot -1
Active beacons: 3
Successful requests: 3

Release and reuse
Released slot 1: true
Strength after release: 0
Request 75 -> slot 1
Active beacons: 3
Successful requests: 4

Pool B
Pool B active: 0
Pool B request 25 -> slot 0
Pool A active: 3
Pool B active: 1
Successful requests: 5
```

Your two-hour route

Stage	What you will do	Time
01	Create the project and files from scratch	15 min
02	Connect Receiver and Beacon with a forward declaration	20 min
03	Create BeaconPool composition and friendship	15 min
04	Implement beacon state and request logic	25 min
05	Implement release, queries and the shared tally	20 min
06	Compare two pools and shared/static state	15 min
07	Final output, troubleshooting and reflection	10 min

01 · Create the project from scratch

Create a new C++ Empty Project named BeaconPoolLab. Use the same solution name unless your lecturer tells you otherwise.

Step 1 - Select the build configuration. Choose Debug and x64 in the Visual Studio toolbar.

Step 2 - Set the project properties. For All Configurations, set Platform Toolset to v145, C++ Language Standard to ISO C++17, and Warning Level to Level 4 (/W4).

Step 3 - Create Main.cpp. Add a C++ source file under Source Files and name it Main.cpp.

Start with one working line

```
#include <iostream>

int main()
{
    std::cout << "Beacon pool lab\n";
    return 0;
}
```

Build the solution and run it before creating any classes.

Expected output

```
Beacon pool lab
```

Checkpoint 1. Do not continue until the empty project builds and runs. This gives you a known working starting point.

Create the remaining files now

Add the following empty files to the same project. Do not create another main function.

Header Files	Source Files
Beacon.h	Beacon.cpp
Receiver.h	Receiver.cpp
BeaconPool.h	BeaconPool.cpp

Work in short cycles. Finish one definition, build, and only then continue. If the linker says a definition is missing, do not hide the problem by removing the call.

02 · Connect Receiver and Beacon

Receiver needs to read a Beacon in one member function, but it does not store a Beacon by value. In Receiver.h, the compiler only needs to know that the class name Beacon exists. This is a good place for a forward declaration.

Create Beacon.h

Type this interface. The private default constructor and private state-changing functions will later be used by BeaconPool through friendship.

```
#ifndef BEACON_H
#define BEACON_H

class BeaconPool;

class Beacon
{
    friend class BeaconPool;

public:
    explicit Beacon(int t_x);
    int x() const;
    int strength() const;
    bool active() const;

private:
    Beacon() = default;
    void activate(int t_strength);
    void reset();

    int m_x{0};
    int m_strength{0};
    bool m_active{false};
};

#endif
```

Create Receiver.h

```
#ifndef RECEIVER_H
#define RECEIVER_H

class Beacon;

class Receiver
{
public:
    explicit Receiver(int t_x);
    int rangeTo(Beacon const& t_beacon) const;

private:
    int m_x;
};

#endif
```

Why this works. The declaration `class Beacon;` supplies the type name. A const-reference parameter does not embed a Beacon inside Receiver, so Receiver.h does not need Beacon's layout.

Write the definitions yourself

Open `Beacon.cpp` and `Receiver.cpp`. Write the definitions described below. The function bodies are not supplied.

Definition	Required behaviour
<code>Beacon::Beacon(int t_x)</code>	Initialise <code>m_x</code> directly from <code>t_x</code> using a member initialiser list.
<code>Beacon::x() const</code>	Return the stored <code>x</code> coordinate.
<code>Beacon::strength() const</code>	Return the stored signal strength.
<code>Beacon::active() const</code>	Return whether the beacon is active.
<code>Receiver::Receiver(int t_x)</code>	Initialise <code>m_x</code> directly from <code>t_x</code> .
<code>Receiver::rangeTo(...) const</code>	Return the non-negative distance between the receiver <code>x</code> and beacon <code>x</code> .

Your `Receiver.cpp` will need both `Receiver.h` and `Beacon.h` because the function body calls `t_beacon.x()`. You may include `<cstdlib>` and use `std::abs` for the integer difference.

Add the connection check to Main.cpp

Add `#include "Beacon.h"` and `#include "Receiver.h"`. Then add these statements after the heading:

```
std::cout << "\nConnection check\n";
Beacon demoBeacon{10};
Receiver receiver{35};
std::cout << "Range to beacon: " << receiver.rangeTo(demoBeacon) << '\n';
```

Expected output

```
Connection check
Range to beacon: 25
```

Checkpoint 2. If the range is 25, your forward-declaration relationship and the two source-file definitions are working.

03 · Compose a BeaconPool

The pool relationship is different. BeaconPool owns three complete Beacon objects by value. The compiler must know the full Beacon layout when it processes the array member, so BeaconPool.h must include Beacon.h. A forward declaration alone is not enough here.

Create BeaconPool.h

```
#ifndef BEACON_POOL_H
#define BEACON_POOL_H

#include "Beacon.h"

class BeaconPool
{
public:
    static constexpr int CAPACITY{3};

    BeaconPool() = default;
    int request(int t_strength);
    bool release(int t_index);
    int activeCount() const;
    int strengthAt(int t_index) const;
    static int successfulRequests();

private:
    Beacon m_beacons[CAPACITY];
    static int s_successfulRequests;
};

#endif
```

Read the relationship before writing any logic

Part	What it means
Beacon m_beacons[CAPACITY];	The pool owns three actual Beacon subobjects. They are not pointers or empty slots.
friend class BeaconPool;	Beacon deliberately allows BeaconPool to call its private constructor and private state-changing operations.
static int s_successfulRequests;	One tally belongs to the BeaconPool class and is shared by every BeaconPool object.
CAPACITY	A class-wide constant. Each pool still owns its own three Beacon objects.

Important distinction. Friendship grants private access. It does not provide Beacon's size or create ownership. The by-value array is the reason BeaconPool.h includes Beacon.h.

Start BeaconPool.cpp

Include BeaconPool.h. Then define the non-inline static member once in the source file using this syntax:

```
#include "BeaconPool.h"

int BeaconPool::s_successfulRequests{0};
```

Do not write a second copy of this definition in the header. In C++17 there are other valid static-definition styles, but this lab uses the two-file form shown in the lecture.

04 · Define the Beacon state changes

Before the pool can manage beacons, finish the two private state-changing functions declared in Beacon.h. Write them in Beacon.cpp.

Function	Required result
activate(int t_strength)	Store t_strength in m_strength and mark m_active as true.
reset()	Set m_strength back to 0 and mark m_active as false.

These functions are private because ordinary client code does not control pooled beacon state directly. BeaconPool is the named friend that may use them.

Implement BeaconPool::request()

Now write the request definition in BeaconPool.cpp. Do not copy a completed function from the tutorial. Use the rules below to build the control flow yourself.

Rule	Required behaviour
1 · Validate	Valid signal strength is 1 to 100 inclusive. For anything else, return -1 and change nothing.
2 · Search	Inspect the slots from index 0 upwards. Choose the first Beacon whose active() query is false.
3 · Activate	Call that Beacon's private activate(t_strength) operation.
4 · Shared tally	Increase s_successfulRequests only after a slot has actually been activated.
5 · Return	Return the chosen index. If all three slots are active, return -1.

Useful building blocks. An indexed `for` loop can run from 0 while `i < CAPACITY`. You can inspect a slot with `m_beacons[i].active()`. The control logic that combines these pieces is your task.

Check request before adding release

Include `BeaconPool.h` in `Main.cpp`. Add the following code after the connection check:

```
std::cout << "\nPool A\n";
BeaconPool poolA{};
std::cout << "Start active: " << poolA.activeCount() << '\n';

int first{poolA.request(30)};
int second{poolA.request(60)};
int third{poolA.request(90)};
int full{poolA.request(50)};

std::cout << "Request 30 -> slot " << first << '\n';
std::cout << "Request 60 -> slot " << second << '\n';
std::cout << "Request 90 -> slot " << third << '\n';
std::cout << "Request 50 -> slot " << full << '\n';
```

The first line calls `activeCount()`, which you will implement on the next stage. If you have not written it yet, comment out that one print statement temporarily rather than changing the interface.

Expected request results

```
Request 30 -> slot 0
Request 60 -> slot 1
Request 90 -> slot 2
Request 50 -> slot -1
```

05 · Release, inspect and count

Complete the remaining BeaconPool definitions. Each function has a small contract. Work through them one at a time and build after each definition.

A · release(int t_index)

Step	Definition requirement
Validate the index	If the index is below 0 or at least CAPACITY, return false before accessing the array.
Check the state	If that Beacon is already inactive, return false.
Reset	Call the Beacon's private reset() function.
Report success	Return true.

B · activeCount() const

Count how many of this pool's three Beacon objects currently report active() == true. Return that count. This query must not activate or release anything.

C · strengthAt(int t_index) const

If the index is invalid, return -1. Otherwise return the selected Beacon's strength(). A released slot should therefore report 0 because reset() cleared it.

D · successfulRequests()

This is a static member function. Return the class-wide s_successfulRequests tally. It has no implicit current BeaconPool object.

Static reminder. Do not try to read `m_beacons` without a particular pool inside `successfulRequests()`. The static function has no `this` receiver. It only needs the static tally.

Add release and reuse to Main.cpp

Near the start of `main()`, add `std::cout << std::boolalpha;` so Boolean values print as true/false. After the Pool A request code, add:

```
std::cout << "Active beacons: " << poolA.activeCount() << '\n';
std::cout << "Successful requests: " << BeaconPool::successfulRequests() << '\n';

std::cout << "\nRelease and reuse\n";
bool released{poolA.release(second)};
std::cout << "Released slot " << second << ": " << released << '\n';
std::cout << "Strength after release: " << poolA.strengthAt(second) << '\n';

int reused{poolA.request(75)};
std::cout << "Request 75 -> slot " << reused << '\n';
std::cout << "Active beacons: " << poolA.activeCount() << '\n';
std::cout << "Successful requests: " << BeaconPool::successfulRequests() << '\n';
```

Expected result

```
Active beacons: 3
Successful requests: 3

Release and reuse
Released slot 1: true
Strength after release: 0
Request 75 -> slot 1
Active beacons: 3
Successful requests: 4
```

Checkpoint 3. Slot 1 should be reused because your request search starts at index 0 and selects the first free slot.

06 · Compare two pools with one shared tally

A static data member belongs to the class rather than to one object. The Beacon objects inside each pool are different: poolA and poolB each own their own array. The successful-request tally is shared across both pool objects.

Predict before running

Question	Your prediction
How many active beacons does a new poolB contain?	_____
Which slot will poolB.request(25) use first?	_____
Will that request change poolA.activeCount()?	_____
What will the shared successful-request tally become?	_____

Add this experiment to Main.cpp

```
std::cout << "\nPool B\n";
BeaconPool poolB{};
std::cout << "Pool B active: " << poolB.activeCount() << '\n';

int other{poolB.request(25)};
std::cout << "Pool B request 25 -> slot " << other << '\n';
std::cout << "Pool A active: " << poolA.activeCount() << '\n';
std::cout << "Pool B active: " << poolB.activeCount() << '\n';
std::cout << "Successful requests: " << BeaconPool::successfulRequests() << '\n';
```

Expected output

```
Pool B
Pool B active: 0
Pool B request 25 -> slot 0
Pool A active: 3
Pool B active: 1
Successful requests: 5
```

What this demonstrates. The arrays are per-object state: using poolB does not occupy poolA. `s_successfulRequests` is class-wide state: successful requests from either pool change the same tally.

One small boundary check

Without adding a new test framework, temporarily request strength 0 from a fresh BeaconPool. Predict the result before running. It should return -1, activeCount() should remain 0, and the shared tally should not increase. Restore your final Main.cpp afterwards.

07 · Put Main.cpp into its final form

Your class logic should now be complete. Main.cpp is only a driver: it creates objects, calls their public interfaces and prints the visible state. Final expected output

```
Beacon pool lab

Connection check
Range to beacon: 25

Pool A
Start active: 0
Request 30 -> slot 0
Request 60 -> slot 1
Request 90 -> slot 2
Request 50 -> slot -1
Active beacons: 3
Successful requests: 3

Release and reuse
Released slot 1: true
Strength after release: 0
Request 75 -> slot 1
Active beacons: 3
Successful requests: 4

Pool B
Pool B active: 0
Pool B request 25 -> slot 0
Pool A active: 3
Pool B active: 1
Successful requests: 5
```

Core lab complete. Your program does not need any extra menu, user input, assertions or student-written test framework. The supplied driver output is the core verification for this practical.

If something is not working

Symptom	First thing to inspect
Receiver.cpp says Beacon is incomplete	Check that Receiver.cpp includes Beacon.h as well as Receiver.h.
BeaconPool.h cannot create the array	Check that BeaconPool.h includes Beacon.h and that Beacon names BeaconPool as a friend.
Unresolved external symbol for a member function	Check that the declaration and definition match and that the .cpp file is part of the project.
Unresolved external for s_successfulRequests	Check for exactly one int BeaconPool::s_successfulRequests{0}; definition in BeaconPool.cpp.
The fourth request returns a real slot	Check that request stops only when it finds an inactive Beacon and returns -1 when all are active.
Released strength is not 0	Check that reset clears both m_active and m_strength.
Pool B changes Pool A occupancy	Check that m_beacons is an ordinary per-object member, not static.

Short understanding check

Question	Be ready to explain
Why can Receiver.h use class Beacon;?	It stores no Beacon by value; the signature only needs the name for a reference.
Why must BeaconPool.h include Beacon.h?	Its fixed array contains complete Beacon subobjects, so their layout is required.
What does friend class BeaconPool permit?	It grants BeaconPool access to Beacon private members; it does not create ownership.
Why is poolA independent from poolB?	Each BeaconPool object owns its own m_beacons array.
Why is successfulRequests shared?	s_successfulRequests is static and belongs to the class rather than one pool object.

Final checklist

- ☐ The solution and BeaconPoolLab project were created from scratch.
- ☐ The project uses v145, C++17, /W4, Debug and x64.
- ☐ Receiver.h uses a forward declaration of Beacon rather than including Beacon.h unnecessarily.
- ☐ Receiver.cpp includes the complete Beacon definition before calling Beacon members.
- ☐ BeaconPool.h includes Beacon.h because it stores Beacon objects by value.
- ☐ BeaconPool owns exactly three Beacon objects in its fixed array.
- ☐ request accepts strengths from 1 to 100, uses the first free slot and returns -1 when it cannot serve the request.
- ☐ release validates the index before array access, rejects an inactive slot and resets released state.
- ☐ activeCount reports this pool only.
- ☐ successfulRequests reports the one shared tally across all pools.
- ☐ The final output matches the published console output.
- ☐ Temporary boundary-check changes have been removed and the final project builds cleanly.

Optional stretch · add full()

Only attempt this after the core output matches. Add the following public query to BeaconPool:

```
bool full() const;
```

Write the definition yourself. It should return true exactly when every slot in this particular pool is active. Do not change the static request tally. Add one print before the first request, one after three successful requests and one after releasing a slot.

Moment	Expected result
New pool	false
After three successful requests	true
After one release	false